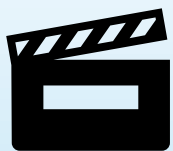




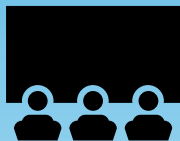
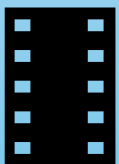
PyData Boston 2025

**Fun With Python and Em😊ji:
What Might Adding Pictures
to Text Programming
Languages Look Like?**



TODAY'S SCRIPT

- 1 THE BIG IDEA
- 2 A PICTURE IS WORTH 1,000 WORDS, RIGHT?
- 3 SO, WHAT MIGHT "EMOJICODE" LOOK LIKE?
- 4 CUTE, BUT IS IT USEFUL?
- 5 A RUDIMENTARY EMOJI-TO-TEXT PREPROCESSOR
- 6 THAT'S ALL FOLKS! (CONCLUSION, Q&A)



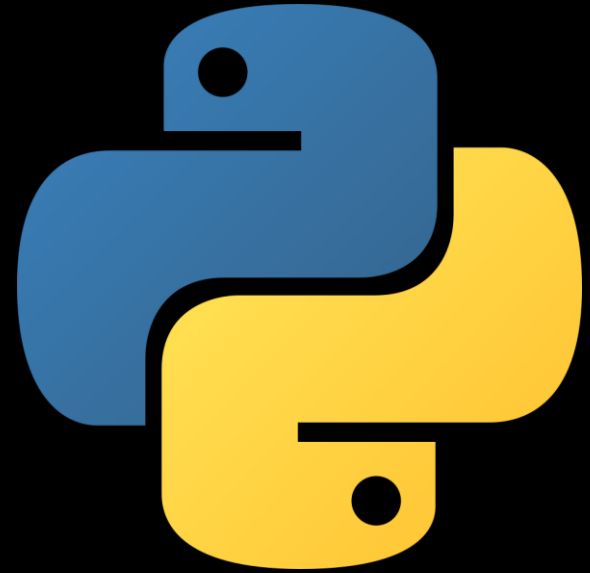
You see things; and
you say "Why?" But I
dream things that
never were; and I say
"Why not?"



George Bernard Shaw
Irish Playwright

1856 - 1950

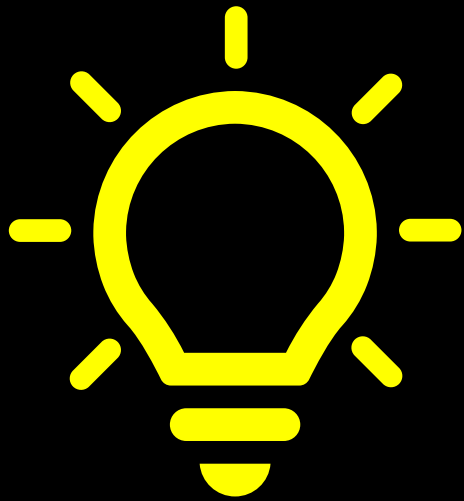
QUOTEHD.COM



+



Emojis in code? Where did that
Aha! moment come from?



COPILOT-GENERATED SAS CODE

```
proc print data=sashelp.cars(obs=5);  
run;
```

```
proc freq data=sashelp.cars;  
tables type;  
run;
```

```
proc means data=sashelp.cars;  
var mpg_city;  
run;
```

```
proc univariate data=sashelp.cars;  
var horsepower;  
run;
```

```
proc sgplot data=sashelp.cars;  
scatter x=weight y=mpg_city;  
run;
```



PROBLEM: How can I stop all those pesky “**RUN;**” statements from cluttering up SAS code but leave the functionality intact for those who wish to use it?



SOLUTION: Replace the “**RUN;**” statements (and the “**;**” from the preceding statement) with an emoji, icon, image, or other pictogram that gets expanded to “**;; RUN;**”



= “**;; RUN;**”

ABC + 😊!

DEBATE CLUB

ABC!

If you enjoy communicating and would like to improve your critical thinking skills **join debate club!**

Here we will learn how to debate,
be persuasive and logically
structure your thoughts.

When? Every Wednesday at 15.30
Where? Room 4A



≡ Would You Use a "Running Semicolon" Character?

📅 Posted 02-20-2022 04:06 PM (3248 views)



"Running Semicolon"



As HBO's Silicon Valley amusingly showed us, programmers can get into heated arguments over the use of [Tabs vs. Spaces](#).

In the SAS world, debates can arise over [whether every DATA step and PROC should be terminated with a RUN or QUIT statement](#) - large SAS programs may include 100+ "RUN;" statements - many technically 'unnecessary' (depending on who you ask!) - making some SAS programmers happy while irritating others. So, a question for lovers and haters of the RUN-even-if-unnecessary rule: If a "running semicolon" or "RUN character" was available, would you be okay with the use of that? 👍👎

Join
Ha

Cal
sou
pro
the
31st

How

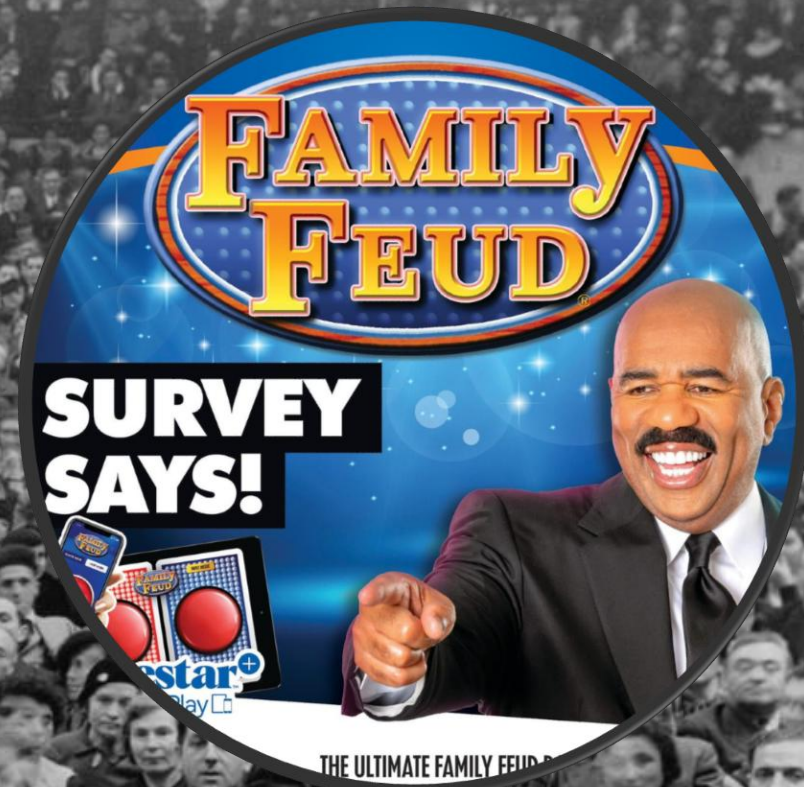
Learn
SAS t
variab

CO
IN S

“I cringe at the notion...”

“Bad idea IMO...”

“Programming languages shouldn’t have decoration: save that for places like Twitter...”



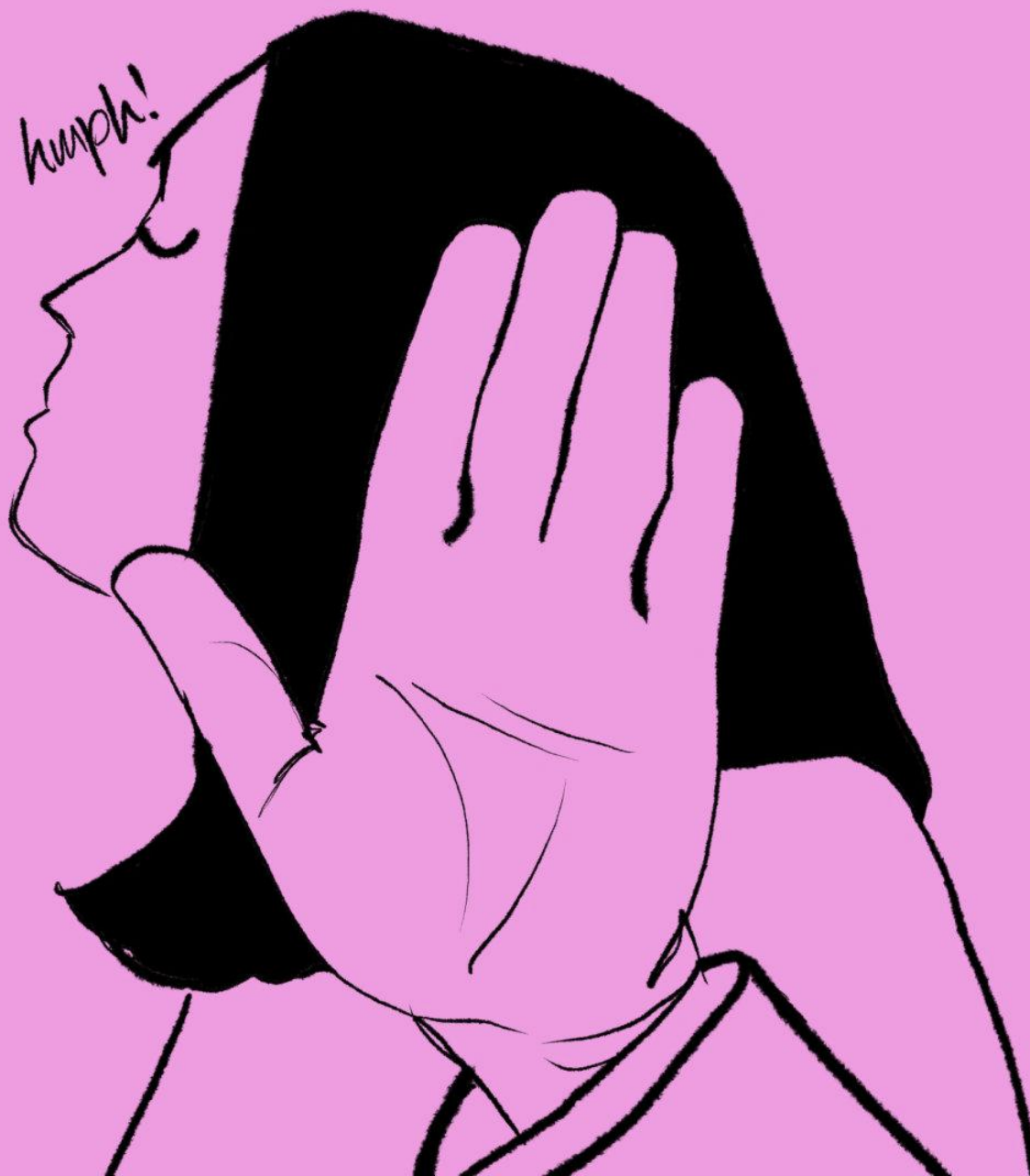
WHY THE PUSHBACK?

GANESH
SUBRAMANIAN



SYMBOLS
A PICTURE IS WORTH A THOUSAND WORDS

Chinmayi Bhat *



IT'S A
TIME-TESTED
CONCEPT



Aa
apple



Bb
bear

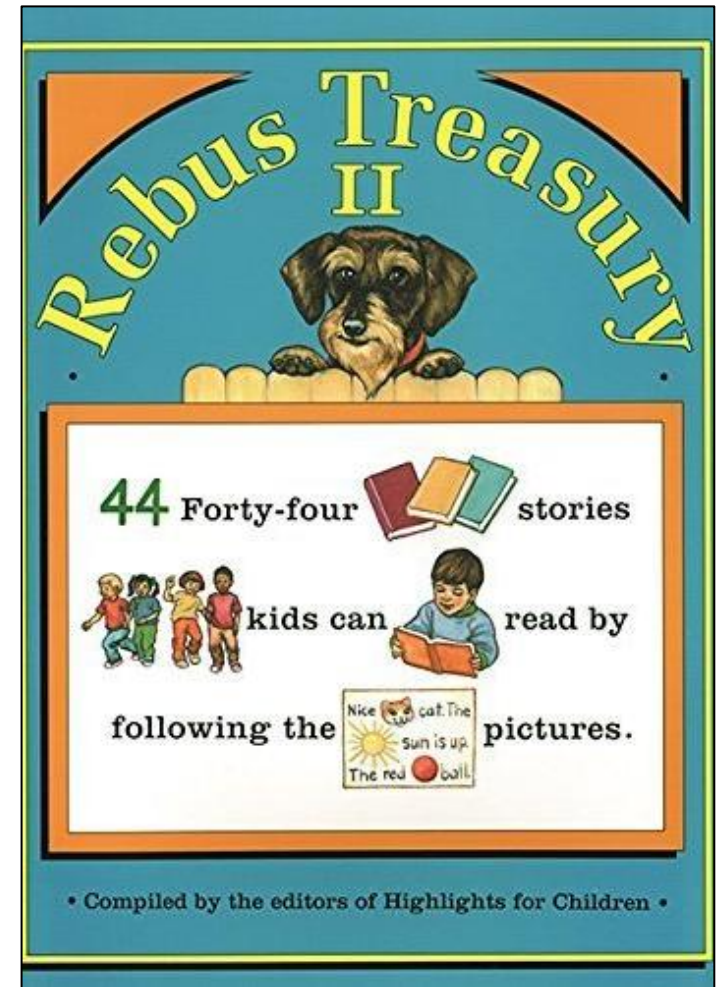
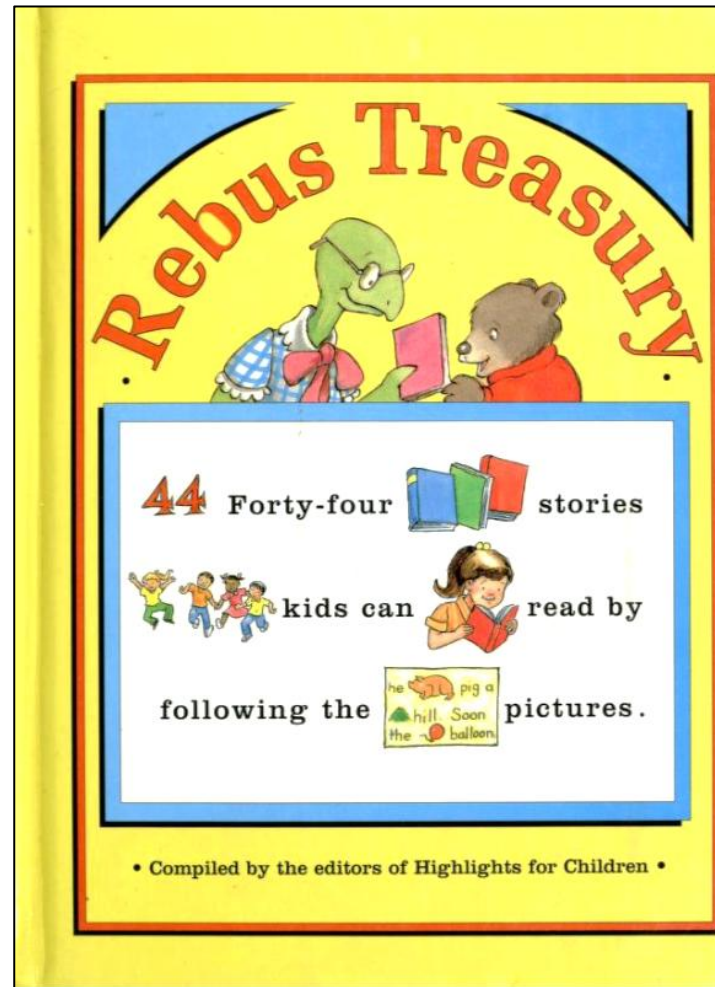


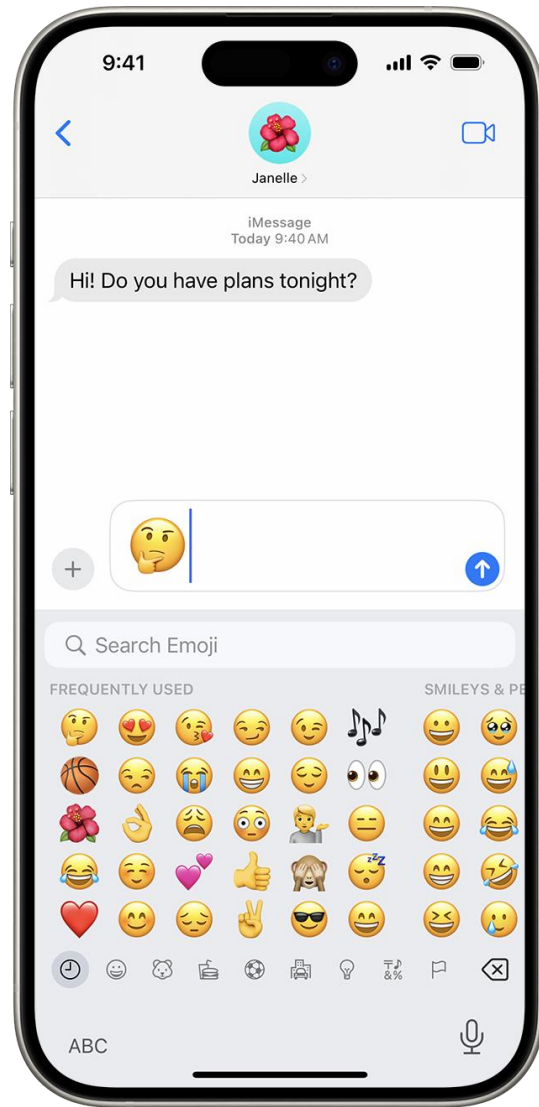
**Children learn to
associate words
and pictures at an
early age**



Highlights for Children: Rebus Stories

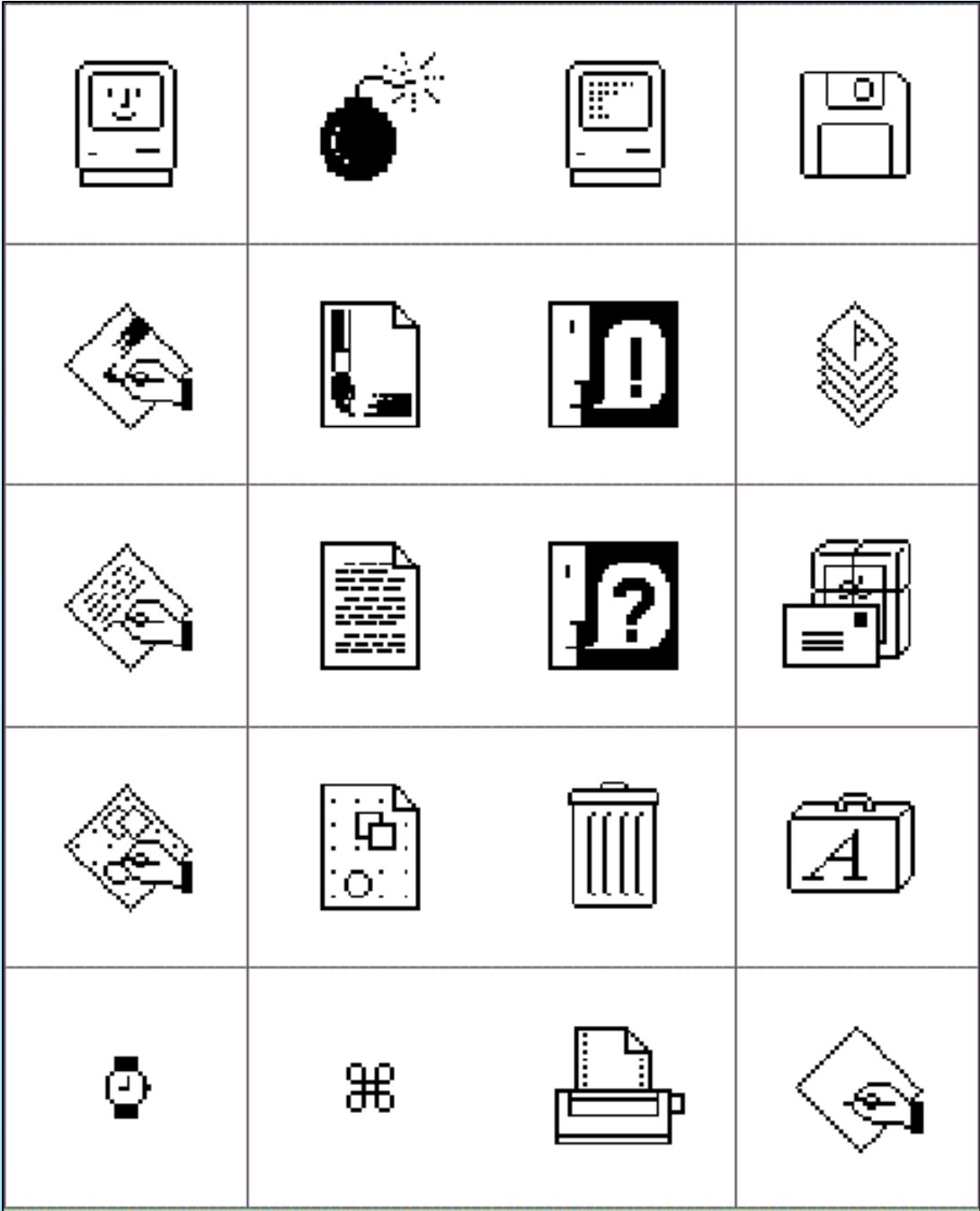
- Use pictures or symbols to represent words or parts of words
- Designed for children just beginning to read
- Playful mix of text and images keeps reading fun, reduces frustration
- Visually engaging reading experiences





I ❤️ NY®

Baa, baa .
Have you any  ?
Yes, sir. Yes, sir.
   full.



Characters Generated by
Cairo Type Font

~	1	2	3	4	5	6	7	8	9	0	-	=	Back Space
🍌	🌴	👁	🪑	🍷	🍰	🍷	🌙	⭐	🌊	✂	➡	⬅	
Tab	a	w	e	r	t	y	u	i	o	p	f	j	\
	✈	☘	🐢	⚡	⚓	📱	💣	🍷	🌍	👉	⬇	🐉	🖨
Caps Lock	a	s	d	f	g	h	j	k	l	;	'	Return	
	✍	⚡	✳	🔑	🌀	➡	📱	🔑	🚗	📱	📱		
Shift	z	x	c	v	b	n	m	,	.	/	?	Shift	
	🐕	✚	🦋	💡	👓	👤	👤	📱	📱	📱	📱		
Opt	🍌											Enter	Opt

~	!	@	#	\$	%	&	*	(	)	-	+	Back Space	
													
Tab	Q	W	E	R	T	Y	U	I	O	P	{	}	
													
Caps Lock	A	S	D	F	G	H	I	K	L	;	"		Return
Lock													
Shift	Z	X	C	V	B	N	M	<	>	?			Shift
													
Opt												Enter	Opt

To Insert These Characters in Your Text
Select "Cairo" Font, Size 18 Point



life $\leftarrow\{\uparrow 1 \omega V.\wedge 3 4=+/,^{-} 1 0$
 $1^{\circ}.\Theta^{-} 1 0 1^{\circ}.\Phi\subset\omega\}$

APL – A PROGRAMMING
 LANGUAGE (Early 1960s)

**THREE
YEARS
LATER...**



Paper 184-2025

Fun With SAS® and Emoji

What Might a Rebus-Influenced Programming
Language Look Like?

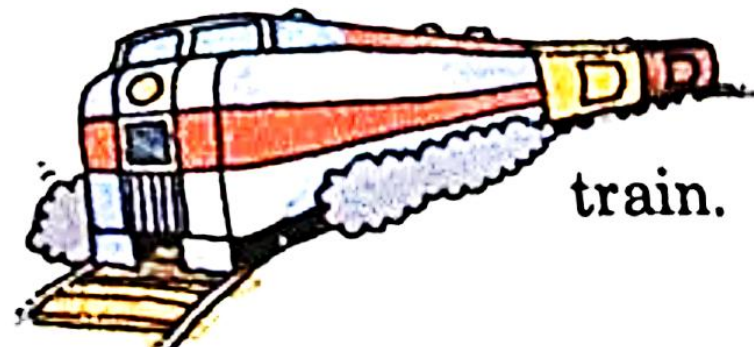
Highlight

Jimmy Goes to the Library

By Pat Kite



Jimmy wanted a book about a



train.

"Let's go to the library,"



Mother said.

They took a



bus to the library.

and the book I want?"



Jimmy asked.



if g=h then i=j; else k=substr(L, 1, 3);



g=h



i=j;



k=



(L, 1, 3);

IF-THEN-ELSE

SAS Code (Text)

```
data test;  
set;  
if a=b then c=d; else e=f;  
if g=h then i=j; else k=substr(L, 1, 3);  
dt=date(); tm=time();  
proc print; run;
```

SAS Code (Text+Emoji)

```
💾 test;  
set;  
🤔 a=b ✅ c=d; ❌ e=f;  
❓ g=h 👍 i=j; 👎 k=✂️ (L, 1, 3);  
dt=📅 (); tm=🕒 ();  
proc 🖨️ ; 🏃
```

SQL

select make from cars group by 1 order by 1 desc;



make



cars

Σ

1



1



;

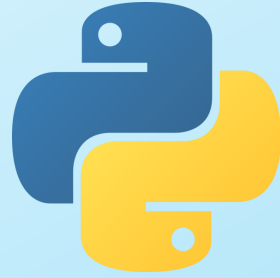
SELECT

SQL Code (Text)

```
proc sql noprint;
create table car_summary as
select make, model,
max(dt) as maxdate format=date10.,
min(tm) as mintime format=time8.
from sashelp.cars t1
join dt_tm t2 on 1=1
group by make, model
order by make, model desc;
```

SQL Code (Text+Emoji)

```
proc sql noprint;
🗄 car_summary as
🛒 make, model,
max(dt) as max 📅 format=📅 10.,
min(tm) as min 🕒 format=🕒 8.
👉 sashelp.cars t1
🔗 dt_tm t2 on 1=1
Σ make, model
📶 make, model 📶 ;
```



```
if datetime.now().weekday() in (5, 6): print ("Weekend!")
```



```
.now().weekday() in (5, 6):
```



```
("Weekend!")
```

IF-THEN-ELSE

Python Code (Text)

```
import pandas, duckdb
Import plotly.express as plotly
from datetime import datetime
print(datetime.now().strftime("%Y-%m-%d"))
if datetime.now().weekday() in (5, 6):
    print ("It's the weekend!")
else:
    print("It's a work day!")

df_cars=pandas.read_csv('cars.csv')
plotly.bar(cars_summary, x='Type',
y='Avg_MPG_City').show()
```

Python Code (Text+Emoji)

```
import 🐼 , 🦆 , 📊 .express as 📊

from 📅 ⌚ import 📅 ⌚
🖨️ ( 📅 ⌚ .now().strftime("%Y-%m-%d"))

🤔 📅 ⌚ .now().weekday() in (5, 6):
    🖨️ ("It's the weekend!")
🚫 :
    🖨️ ("It's a work day!")

📊 .bar( 🚗 _summary, x='Type',
y='Avg_MPG_City').show()
```

SQL

duckdb.sql("select type, avg(MPG_City) from cars group by 1 order by 2 desc")

 .sql("  type, avg(MPG_City) from  Σ 1  2  ")

SELECT (DUCKDB)

SQL Code (Text)

```
import pandas, duckdb
df_cars=pandas.read_csv('cars.csv')
cars_summary=duckdb.sql("""
select type,
avg(MPG_City) as avg_MPG_City,
avg(MSRP) as avg_MSRP
from df_cars
group by 1 order by 2 desc, 1
""").df()
print(cars_summary)
```

SQL Code (Text+Emoji)

```
import 🐼 , 🦆
df_🚗 = 🐼 .read_csv('🚗.csv')
🚗_summary= 🦆 .sql("""
🛒 type,
avg(MPG_City) as avg_MPG_City,
avg(MSRP) as avg_MSRP
from df_🚗
Σ 1 2 ⬆️ 2 ⬇️ , 1
""").df()
🖨️ ( 🚗 _summary)
```

WHERE'S THE BEEF?,"



**“Cute, but are there
any real benefits?”**

VS CODE SAMPLE CODE SNIPPET

```
{ "Insert Stuff": {  
    "prefix": "insline",  
    "body": ["# Connect to Snowflake",  
"conn = snowflake.connector.connect(",  
"account='your_account',",  
"user='your_username',",  
"password='your_password')", ],  
    "description": "Insert lines" }}
```

VS CODE SAMPLE CODE SNIPPET

```
{  "Insert Stuff": {  
    "prefix": "insline",  
    "body": ["# Connect to Snowflake",  
"conn = snowflake.connector.connect(",  
"account='your_account'",  
"user='your_username'",  
"password='your_password')", ],  
    "description": "Insert lines" }}
```

EMOJI-MAPPED CODE SNIPPET

```
'🔌': '''  
# Connect to Snowflake  
conn = snowflake.connector.connect(  
    account='your_account',  
    user='your_username',  
    password='your_password')  
'''
```

EMOJI-MAPPED CODE SNIPPET

```
'🔌': '''  
# Connect to Snowflake  
conn = snowflake.connector.connect(  
    account='your_account',  
    user='your_username',  
    password='your_password')  
'''
```

EMOJI-MAPPED CODE SNIPPET

```
'🔌': '''  
# Connect to Snowflake  
conn = snowflake.connector.connect(  
    account='your_account',  
    user='your_username',  
    password='your_password')  
'''
```

Do you like to use keyboard shortcuts and code snippets? Emoji-mapped code can offer the same keystroke reduction benefit, **PLUS** allows code to be maintained at the single emoji level instead of at the fully-expanded code level – here that means 1 emoji vs. 5 lines of code to connect to a Snowflake database.

The emoji mapping in effect provides a similar kind of functionality as a simple, single character macro.

Do you like to use keyboard shortcuts and code snippets? Emoji-mapped code can offer the same keystroke reduction benefit, **PLUS** allows code to be maintained at the single emoji level instead of at the fully-expanded code level – here that means 1 emoji vs. 5 lines of code to connect to a Snowflake database.

The emoji mapping in effect provides a similar kind of functionality as a simple, single character macro.

Do you like to use keyboard shortcuts and code snippets? Emoji-mapped code can offer the same keystroke reduction benefit, **PLUS** allows code to be maintained at the single emoji level instead of at the fully-expanded code level – here that means 1 emoji vs. 5 lines of code to connect to a Snowflake database.

The emoji mapping in effect provides a similar kind of functionality as a simple, single character macro.

```
from xml.etree.cElementTree import ElementTree, ParseError
def parse_xml_from_file(filename):
    try:
        data = ElementTree().parse(filename)
    except ParseError:
        print('Failed to parse file {}'.format(filename))

filename = r'C:\Data.xml'
parse_xml_from_file(filename)
```

Module name

Class name

Function name

Parameter name

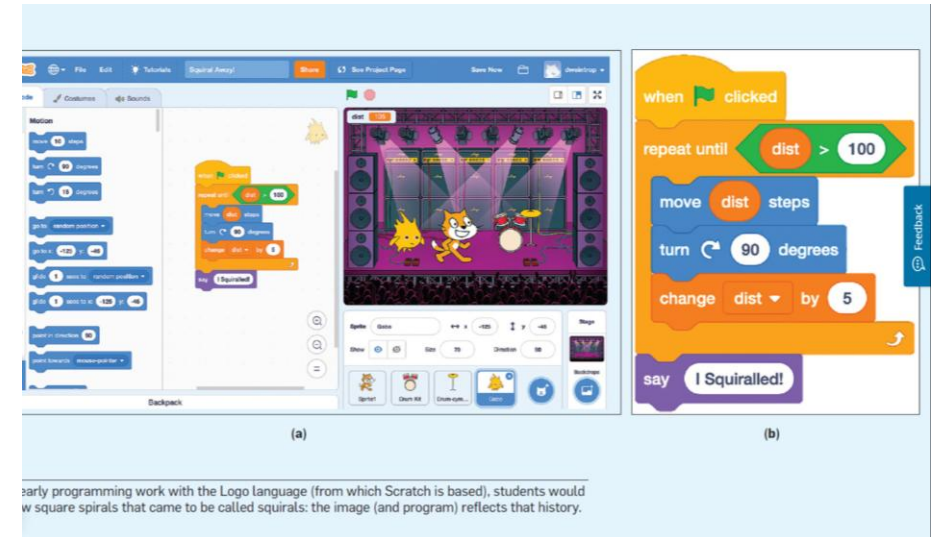
Syntax Highlighting

Like using syntax highlighting and code-naming standards, e.g., CamelCase, Hungarian notation?

Emoji mapped to text could represent all or part of your package names, package aliases, variable names, function names, constants, etc. Same improved readability, **PLUS** emojis will help make names 'pop' even more!

Make Coding More Playful & Approachable!

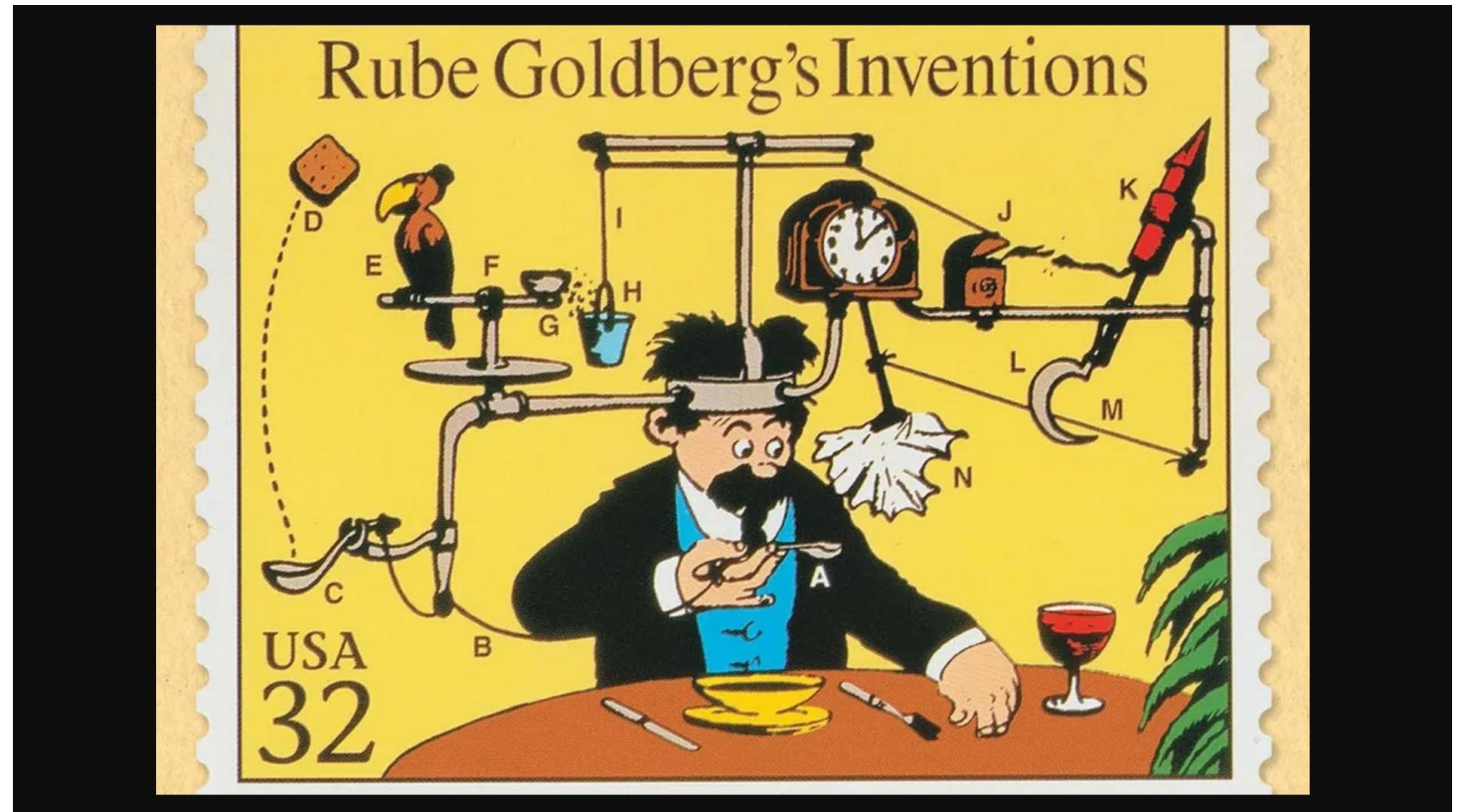
For those new to text programming (e.g., kids), letting them use emoji-to-text mappings – both pre-defined and of their own creation – might help get them to embrace coding, ease the transition from block to text coding, and perhaps even help some grasp algebra concepts before they're deemed "ready" for that subject!



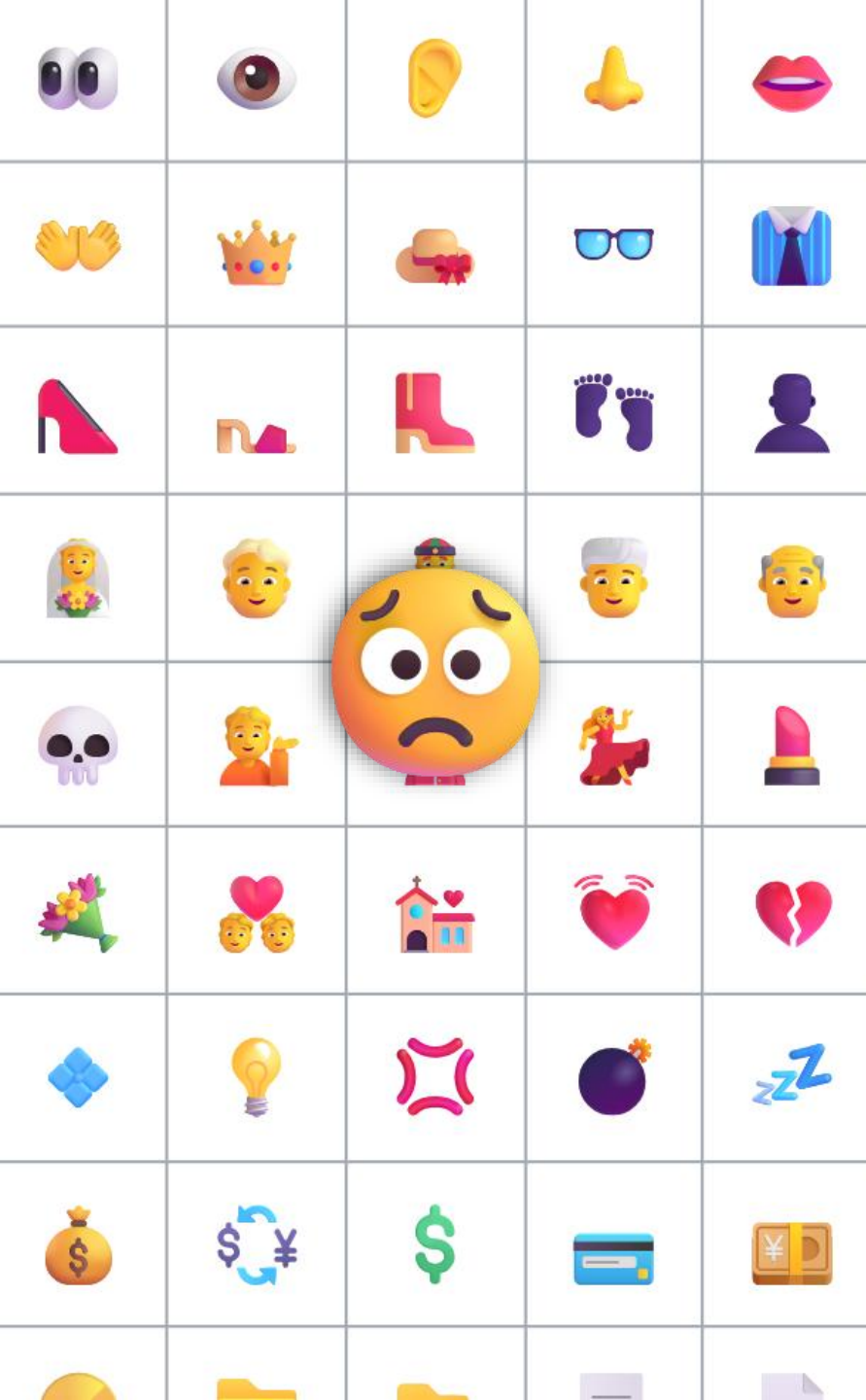
```
src > quizzes > tests > display_tests.py > DisplayQuizViewTests > test_quiz_redirects
4 from django.test import TestCase
5 from django.urls import reverse
6
7
8 class DisplayQuizViewTests(TestCase):
9     def test_quiz_404(self):
10         url = reverse("quizzes:display_quiz", args=(12,))
11         response = self.client.get(url)
12         self.assertEqual(response.status_code, 404)
13
14     def test_quiz_redirects(self):
15         quiz, question, _ = create_quiz()
16         url = reverse("quizzes:display_quiz", args=(quiz.pk,))
17         response = self.client.get(url)
18         self.assertRedirects(response, reverse("quizzes:display_question", args=(quiz.pk, question.pk)))
19
20
21 class DisplayQuestionViewTests(TestCase):
22     def test_quiz_404(self):
23         url = reverse("quizzes:display_question", args=(12, 1))
24         response = self.client.get(url)
25         self.assertEqual(response.status_code, 404)
26
27     def test_question_404(self):
28         quiz, question, _ = create_quiz()
29         url = reverse("quizzes:display_question", args=(quiz.pk, question.pk + 100))
30         response = self.client.get(url)
```

Proof-of-Concept

A Simple Emoji-to-Text Preprocessor



- Translates emoji-laden code into all text for Python
- Uses an IPython input transformer with VS Code + Jupyter Extension
- Just a quick-and-dirty POC, ignores messiness of real-world language design and implementation!



Emojis are complicated!



Man Running Facing Right

Code Points (Sequence)	UTF-8 Encoding (Hex)
U+1F3C3 [] U+200D U+2642 [♂] U+FE0F U+200D U+27A1 [➡] U+FE0F	F0 9F 8F 83 E2 80 8D E2 99 82 EF B8 8F E2 80 8D E2 9E A1 EF B8 8F

Emoji Encoding

- In the Unicode and UTF-8 world of emoji, **one character may now occupy 1-4 bytes**
- Complex graphemes built from **multiple code points, joiners & modifiers** require even more!

Character	Description	Code Points (Sequence)	UTF-8 Encoding (Hex)	Bytes
A	Latin Capital A	U+0041	41	1
🤔	Thinking Face	U+1F914	F0 9F A4 94	4
✅	White Heavy Check Mark	U+2705	E2 9C 85	3
❌	Cross Mark Button	U+274E	E2 9D 8E	3
🏃	Person Running	U+1F3C3	F0 9F 8F 83	4
🏃	Man Running Facing Right (ZWJ seq)	U+1F3C3 [🏃] U+200D U+2642 [♂] U+FE0F U+200D U+27A1 [➡] U+FE0F	F0 9F 8F 83 E2 80 8D E2 99 82 EF B8 8F E2 80 8D E2 9E A1 EF B8 8F	22

Houston, We Have an Emoji Problem



Bad News:

Python has native emoji support but

- 😱 Limited to single codepoint emoji
- 😱 No support for zero-width joiners
- 😱 No support for variation selectors

Good News:

Python packages fill in the gaps

- 😊 Easily identify and extract emojis
- 😊 Supports complex emoji ‘clusters’
- 😊 Use [regex](#) package with /X options

```
import regex, pandas as pd
dict = {'🤔': 'if', '✅': 'then', '❌': 'else', '🏃': 'run;'}
str="🤔 a=b ✅ c=d; ❌ e=f; 🏃 "
df = pd.DataFrame([match.group() for match in regex.finditer(r"\X", str)],
                  columns=['grapheme'])
df['text'] = df['grapheme'].map(dict)
df['text'].fillna(df['grapheme'], inplace=True)
df
```

#	Grapheme	Text	#	Grapheme	Text	#	Grapheme	Text	#	Grapheme	Text
0	🤔	if	6	✅	then	12			18	;	;
1			7			13	❌	else	19		
2	a	a	8	c	c	14			20	🏃	run;
3	=	=	9	=	=	15	e	e			
4	b	b	10	d	d	16	=	=			
5			11	;	;	17	f	f			

A Python Emoji Preprocessor



Python code with emojis



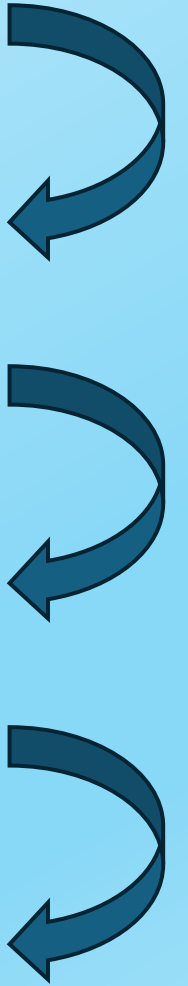
IPython transformer replaces emojis with text (via regex)



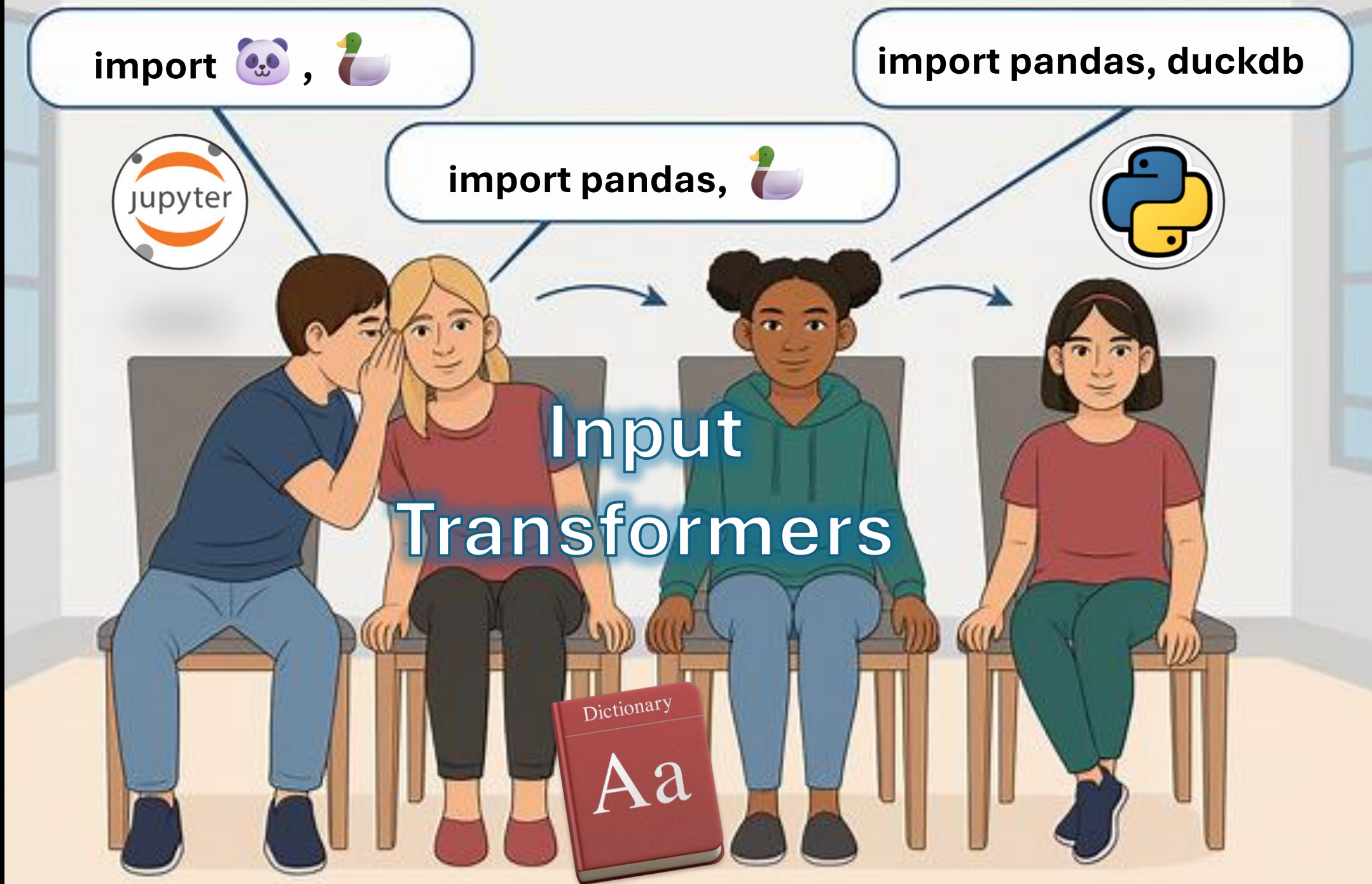
Python code without emojis



Python runs emoji-free code



IPython Custom Input Transformation



```
dict = {'🤔': 'if', '❌': 'else', '💾': 'data',  
        '🐼': 'pandas', '🦆': 'duckdb',  
        '📊': 'plotly', '💾': 'data',  
        '📅': 'date', '🕒': 'time', '🖨': 'print',  
        '🗄': 'create table', '🗑': 'drop table',  
        '🛒': 'select', '⬅': 'from',  
        '⬆': 'asc', '⬇': 'desc',  
        '🔗': 'join', '↕': 'order by',  
        'Σ': 'group by'}
```

Define emoji-to-text mapping, set flag indicating translation is active

```
global dict, emoji2txt
globals()['emoji2txt']=False
```

```
dict = {
'😬': 'if', '❌': 'else', '🖨️': 'print', '🐼': 'pandas', '🦆': 'duckdb', '📊': 'plotly',
'🔤': 'str', '💾': 'data', '📅': 'date', '🕒': 'time', '🔄': 'while',
'🗄️': 'create table', '🗑️': 'drop table', '🛒': 'select', '👈': 'from', '🔗': 'join',
'⬆️': 'order by', '⬇️': 'asc', '⬇️': 'desc', 'Σ': 'group by',
'🚗': 'cars',
'🔌': '"""# Connect to Snowflake
conn = snowflake.connector.connect(
account='your_account',
user='your_username',
password='your_password')
"""}

```

```
globals()['emoji2txt']=True
```

```
import 🐼, 🦢, 📈.express as 📈
from 📅 ⌚ import 📅 ⌚
```

PACKAGES

```
🖨️ ( 📅 ⌚ .now().strftime("%Y-%m-%d"))
🤔 📅 ⌚ .now().weekday() in (5, 6):
    🖨️ ("It's the weekend!")
🟢 ✕ :
    🖨️ ("\nIt's a work day!")
```

PYTHON

```
df_ 🚗 = 🐼 .read_csv(' 🚗 .csv')
```

```
🚗 _summary= 🦢 .sql("""
🛒 type, avg(MPG_City) as Avg_MPG_City, Avg(MSRP) as avg_MSRP
from df_ 🚗 Σ 1 ⬆ 2 ⬇ , 1
""
```

SQL

```
).df()
🖨️ ("\n", 🚗 _summary)
```

```
📈 .bar( 🚗 _summary, x='Type', y='Avg_MPG_City').show()
```

PLOTLY



...

You have not yet opened a folder.

Open Folder

Opening a folder will close all currently open editors. To keep them open, [add a folder](#) instead.

demo4pydata.ipynb ✕

C: > Users > tedco > OneDrive > Pydata2025 >  demo4pydata.ipynb >  # Remove existing input transformer

 Generate
  Code
  Markdown
  Run All
  Restart
  Clear All Outputs
  ...

 base (Python 3.10.9)

▷ ✓

```
# Remove existing input transformer
ip.input_transformers_cleanup.remove(emoji2text)
```

Python

```
# Create global emoji-to-text Python dictionary, set global indicator that translation is active
global dict, emoji2txt
globals()['emoji2txt']=False
dict = {
'😞':'if', '❌':'else', '🖨':'print',
'🐼':'pandas', '🦆':'duckdb', '📈':'plotly', '📊':'excel',
'🔠':'str', '💾':'data', '📅':'date', '🕒':'time', '🔄':'while',
'🗄':'create table', '🗑':'drop table', '🛒':'select', '⬅':'from',
'🔗':'join', '📶':'order by', '⬇':'asc', '⬇':'desc', 'Σ':'group by',
'🚗':'cars'}
globals()['emoji2txt']=True
```

Python

```
import regex, json, pandas as pd
```

> OUTLINE

> TIMELINE



Spaces: 4

{1}

Cell 1 of 4

Ⓐ

⊗₂₄ Δ₇



A GPT-5 Second Opinion

“Give me some pros and cons of using emojis to represent certain statement names, variable names, function names, literals, and code snippets in Python...”

“Create a summary table explaining if each proposed use is Pythonic or not Pythonic...”



GPT-5 Generated “Verdict Table”

Use Case	Possible?	Pythonic?	Notes
String Literals (" 🌍 ")	✅ Yes	✅ Yes	Safe and widely used
Comments with Emojis	✅ Yes	⚠️ Okay	Decorative only; use sparingly
Variable Names (🔥)	✅ Yes	❌ No	Works, but unreadable, flagged by linters
Function Names (🐍)	✅ Yes	❌ No	Technically valid, but violates clarity
Emojis via IPython Transformer	✅ Yes	❌ No	Hacky DSL layer; non-standard
Package Names (🔥 .py)	✅ Yes	❌ No	Technically importable, but breaks portability and readability
Emoji Macros (🔌 → code snippet)	✅ Yes	❌ No	Fun for demos; expands to full code, but non-standard and fragile
Complex Grapheme (🏃)	⚠️ Sometimes	❌ No	Can expand via transformer if exact ZWJ + selectors match; fragile across editors
Emoji in place of Python keywords via input transformer	✅ Yes (IPython only)	❌ No	Possible by preprocessing input; playful DSL trick, but non-standard & not portable

GPT-5 Generated “Verdict Table”

Use Case	Possible?	Pythonic?	Notes
String Literals (" 🌍 ")	✅ Yes	✅ Yes	Safe and widely used
Comments with Emojis	✅ Yes	👉 Okay	Decorative only, use sparingly
Variable Names (🔥)	✅ Yes	❌ No	Works but unreadable, flagged by linters
Function Names (🐍)	✅ Yes	❌ No	Technically valid, but violates clarity
Emojis via IPython Transformer	✅ Yes	❌ No	Tricky DSL layer; non-standard
Package Names (🍌 .py)	✅ Yes	❌ No	Technically importable, but breaks portability and readability
Emoji Macros (🔌 → code snippet)	✅ Yes	❌ No	Fun for demos; expands to full code, but non-standard and fragile
Complex Grapheme (🏃)	⚠️ Sometimes	❌ No	Can expand via transformer if exact ZWJ + selectors match; fragile across editors
Emoji in place of Python keywords via input transformer	✅ Yes (Python only)	❌ No	Possible by preprocessing input; playful DSL trick, but non-standard & not portable

COMMUNICATIONS
of the
ACM

Explore Topics

Learning How Learning Works

Can LLMs learn 'impossible' languages?

By [Jennifer Goforth Gregory](#)
Posted Dec 5 2025



Share

Print

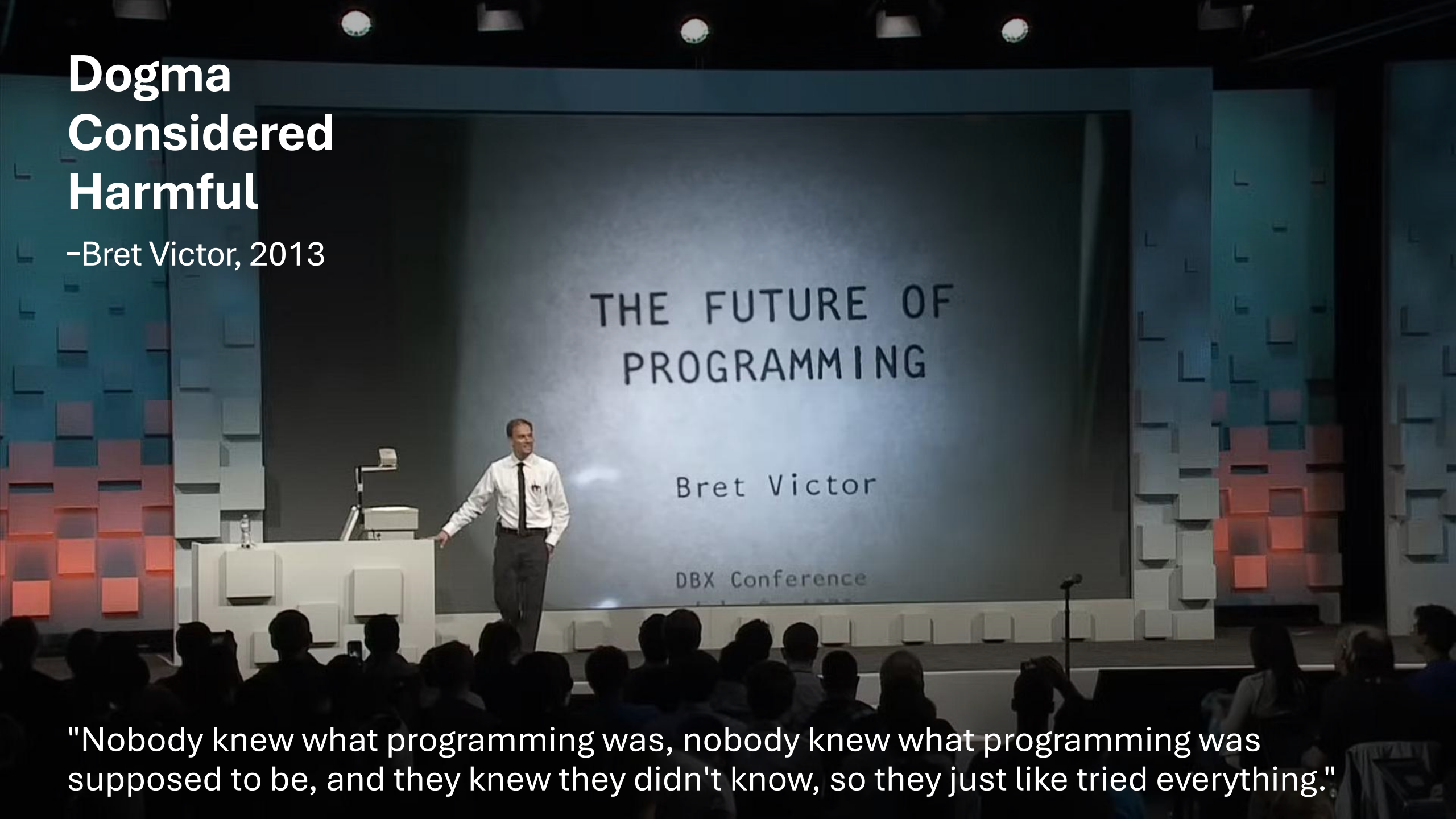
Join the Discussion

View in the ACM Digital Library



Dogma Considered Harmful

-Bret Victor, 2013



THE FUTURE OF
PROGRAMMING

Bret Victor

DBX Conference

"Nobody knew what programming was, nobody knew what programming was supposed to be, and they knew they didn't know, so they just like tried everything."

Don't be
'That Guy'

BINARY

```
1999015201 5801610167  
0170017301 7601820185  
0141014401 5001530156  
0154020402 0501630254  
0211014201 4901720143  
0203620602 1901770235  
0153015502 7201930253
```

Programming

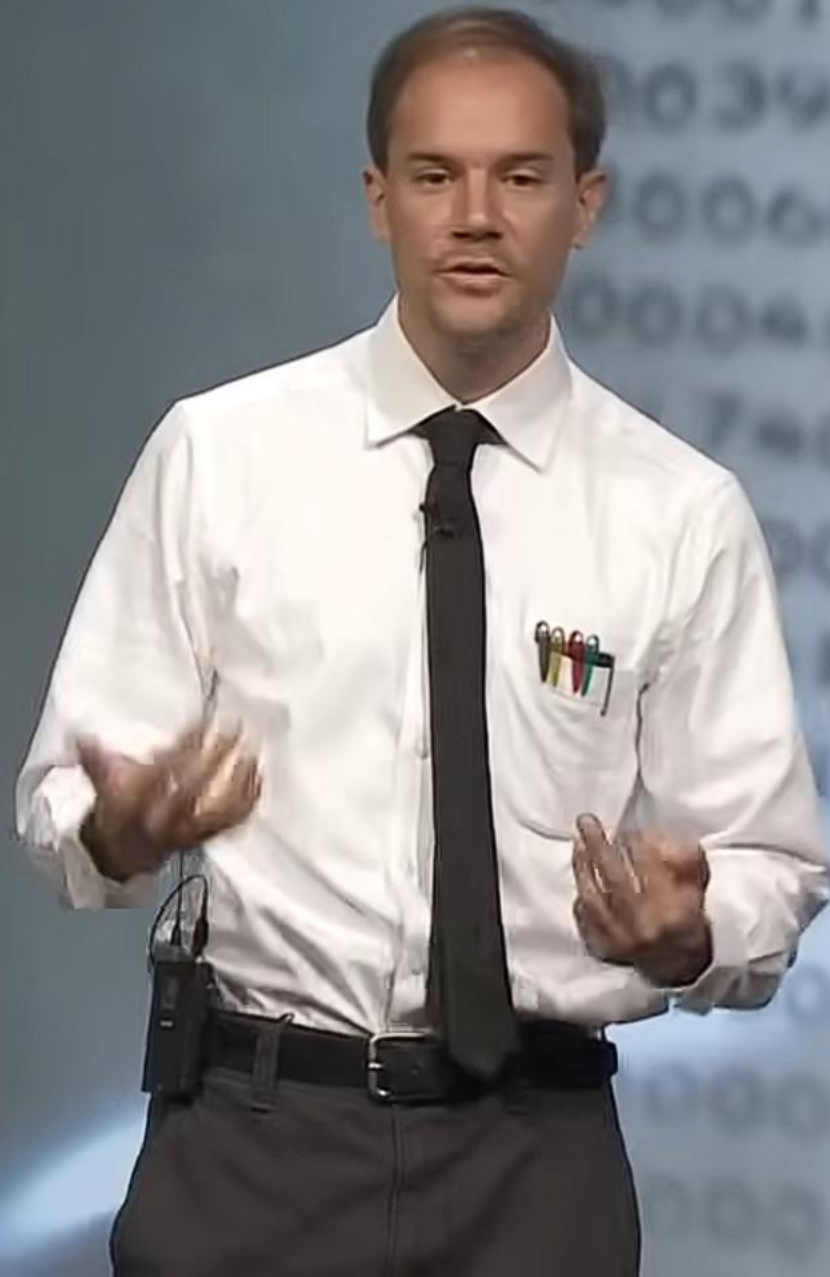
FORTRAN

```
C 0000 RECTANGULAR MATRIX  
C 0000 MULTIPLICATION  
DIMENSION A(4*5), B(5*3)  
READ 1, A+B  
READ 1, N+M  
7 00 4 1 1 N
```

NOT
Programming

```
0000000000 0000000000  
6501640167 6901701822  
6901851822 01680141  
6901961823 00010162  
6800010169 40070211  
1500390143 0040203  
6580060231 90192  
3500040221 58002  
3901740221 00137  
2401100221 0201  
6916618221 0272  
018718221 0010160  
4601460221 0010162  
4601740171 0008000  
00000000 0000000  
00000001 00000269  
00000000 00001999
```

I think you have to say,
“We don’t know what
programming is...”



QUESTIONS?

Ted Conway

Chicago, IL

ted.j.conway@gmail.com

@vivasasvegas X 🦋

